

Checkout Redesign A/B Testing & Experiment Analysis

About the Project

This project analyzes a four-week A/B test conducted to evaluate a redesigned e-commerce checkout experience. The experiment compared the existing checkout flow (**Control**) with a simplified redesigned checkout (**Treatment**) to determine whether the new experience improved customer conversion and revenue performance.

The analysis focuses on validating experiment integrity before measuring results, including assignment consistency, treatment allocation, invalid orders, and data-quality issues. After cleaning the experiment data, the primary metric is **user conversion rate**, defined as the percentage of assigned users who placed at least one valid order after assignment.

In addition to conversion, the project evaluates **Average Order Value (AOV)** and **Revenue per User (RPU)** to understand the redesign's broader financial impact. Statistical hypothesis testing and confidence intervals are used to determine whether observed differences between the control and treatment groups are statistically meaningful.

The analysis also includes follow-up segment analysis across **device type** and **new vs. returning visitors**, while accounting for multiple comparisons. Finally, the project estimates the potential monthly revenue impact of launching the redesigned checkout and provides an evidence-based launch recommendation.

Key Objectives

- Validate the integrity and quality of the A/B test data.
- Identify and handle users or orders that should be excluded from the analysis.
- Measure checkout conversion, AOV, and revenue per user.
- Quantify the treatment effect using confidence intervals and statistical testing.
- Analyze experiment results across important customer segments.
- Estimate the potential revenue impact of a full rollout.
- Provide a clear, data-driven recommendation for the checkout redesign.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from scipy import stats
from scipy.stats import norm
from statsmodels.stats.proportion import proportions_ztest
from statsmodels.stats.proportion import proportion_confint
from statsmodels.stats.multitest import multipletests
from IPython.display import display
import math
import warnings
warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

print("Libraries imported successfully!")

```

Libraries imported successfully!

```

In [3]: df1 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Data Lab Project\Redesign AB Testing\assignments.csv")
df1.head()

```

```

Out[3]:

```

	user_id	variant	assigned_at	device	visitor_type
0	9371	treatment	02-09-2025 00:00	desktop	new
1	14953	control	02-09-2025 00:07	mobile	returning
2	7085	control	02-09-2025 00:08	desktop	new
3	1063	control	02-09-2025 00:08	mobile	new
4	27723	treatment	02-09-2025 00:08	mobile	new

```

In [4]: # Dataset shape
df1.shape

```

```

Out[4]: (31284, 5)

```

```

In [5]: # Column names
df1.columns.tolist()

```

```

Out[5]: ['user_id', 'variant', 'assigned_at', 'device', 'visitor_type']

```

```
In [6]: # Data types and non-null counts
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31284 entries, 0 to 31283
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   user_id         31284 non-null  int64
1   variant         31284 non-null  object
2   assigned_at     31284 non-null  object
3   device          31284 non-null  object
4   visitor_type    31284 non-null  object
dtypes: int64(1), object(4)
memory usage: 1.2+ MB
```

```
In [8]: df1["assigned_at"] = pd.to_datetime(
        df1["assigned_at"],
        format="%d-%m-%Y %H:%M",
        errors="coerce"
    )
```

```
In [9]: df1["assigned_at"].dtype
```

Out[9]: dtype('<M8[ns]')

```
In [10]: # Basic summary
df1.describe(include="all").T
```

Out[10]:		count	unique	top	freq	mean	min	25%	50%	75%	max	std
	user_id	31284.0	NaN	NaN	NaN	15499.709404	1.0	7743.75	15497.5	23252.25	31000.0	8950.560967
	variant	31284	2	treatment	15661	NaN	NaN	NaN	NaN	NaN	NaN	NaN
	assigned_at	31284	NaN	NaN	NaN	2025-09-16 02:38:22.021480704	2025-09-02 00:00:00	2025-09-09 02:01:30	2025-09-16 03:50:30	2025-09-23 03:34:15	2025-10-04 05:36:00	NaN
	device	31284	3	desktop	15282	NaN	NaN	NaN	NaN	NaN	NaN	NaN
	visitor_type	31284	2	new	19300	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
In [11]: # Variant distribution
df1["variant"].value_counts()
```

```
Out[11]: variant
treatment    15661
control      15623
Name: count, dtype: int64
```

```
In [12]: # Device distribution
df1["device"].value_counts()
```

```
Out[12]: device
desktop      15282
mobile       13472
tablet       2530
Name: count, dtype: int64
```

```
In [13]: # Check users assigned to multiple variants
groups_per_user = df1.groupby("user_id")["variant"].nunique()

mixed_users = groups_per_user[groups_per_user > 1]

print("Users assigned to multiple variants:", len(mixed_users))
```

Users assigned to multiple variants: 284

```
In [14]: # Missing values
df1.isnull().sum()
```

```
Out[14]: user_id      0
variant      0
assigned_at    0
device        0
visitor_type   0
dtype: int64
```

```
In [15]: df1.duplicated().sum()
```

```
Out[15]: np.int64(0)
```

```
In [16]: print("Missing dates:", df1["assigned_at"].isna().sum())
print("Start:", df1["assigned_at"].min())
print("End:", df1["assigned_at"].max())
```

Missing dates: 0
Start: 2025-09-02 00:00:00
End: 2025-10-04 05:36:00

```
In [17]: # Check assignments outside the stated experiment window
outside_window = df1[
    (df1["assigned_at"] < "2025-09-02") |
    (df1["assigned_at"] >= "2025-09-30")
]

print("Assignments outside experiment window:", len(outside_window))
```

Assignments outside experiment window: 20

```
In [18]: outside_window.head(10)
```

```
Out[18]:
```

	user_id	variant	assigned_at	device	visitor_type
31264	634	treatment	2025-09-30 00:23:00	desktop	returning
31265	25819	treatment	2025-09-30 00:36:00	mobile	returning
31266	28609	control	2025-09-30 01:55:00	mobile	returning
31267	19384	control	2025-09-30 03:19:00	desktop	returning
31268	27542	control	2025-09-30 11:57:00	mobile	returning
31269	3469	control	2025-09-30 13:39:00	desktop	returning
31270	25949	control	2025-09-30 18:58:00	mobile	returning
31271	30384	treatment	2025-09-30 20:31:00	desktop	returning
31272	9662	control	2025-09-30 20:32:00	mobile	returning
31273	28849	control	2025-09-30 22:24:00	desktop	returning

```
In [19]: df1["assigned_at"].dt.date.value_counts().sort_index()
```

```
Out[19]: assigned_at
2025-09-02    1078
2025-09-03    1146
2025-09-04    1082
2025-09-05    1086
2025-09-06    1080
2025-09-07    1133
2025-09-08    1118
2025-09-09    1135
2025-09-10    1064
2025-09-11    1133
2025-09-12    1120
2025-09-13    1142
2025-09-14    1055
2025-09-15    1106
2025-09-16    1099
2025-09-17    1149
2025-09-18    1094
2025-09-19    1083
2025-09-20    1104
2025-09-21    1149
2025-09-22    1117
2025-09-23    1165
2025-09-24    1081
2025-09-25    1144
2025-09-26    1116
2025-09-27    1187
2025-09-28    1173
2025-09-29    1125
2025-09-30     10
2025-10-01     5
2025-10-02     1
2025-10-03     3
2025-10-04     1
Name: count, dtype: int64
```

```
In [20]: # Identify users assigned to multiple variants
groups_per_user = df1.groupby("user_id")["variant"].nunique()

mixed_users = groups_per_user[groups_per_user > 1].index

print("Mixed-variant users:", len(mixed_users))
```

Mixed-variant users: 284

```
In [21]: mixed_assignments = df1[df1["user_id"].isin(mixed_users)]  
  
        print("Assignment rows belonging to mixed users:", len(mixed_assignments))
```

Assignment rows belonging to mixed users: 568

```
In [22]: mixed_assignments.sort_values(["user_id", "assigned_at"]).head(20)
```

Out[22]:

	user_id	variant	assigned_at	device	visitor_type
8324	7	treatment	2025-09-09 12:26:00	mobile	returning
12120	7	control	2025-09-12 22:35:00	desktop	returning
13268	189	treatment	2025-09-13 23:03:00	desktop	new
16256	189	control	2025-09-16 16:56:00	mobile	returning
3250	260	control	2025-09-04 22:53:00	desktop	returning
4951	260	treatment	2025-09-06 12:20:00	mobile	returning
7615	432	treatment	2025-09-08 21:35:00	desktop	returning
7824	432	control	2025-09-09 02:06:00	mobile	returning
16416	522	treatment	2025-09-16 20:08:00	mobile	new
18634	522	control	2025-09-18 20:04:00	desktop	returning
29171	634	control	2025-09-28 04:16:00	mobile	new
31264	634	treatment	2025-09-30 00:23:00	desktop	returning
642	861	treatment	2025-09-02 14:02:00	desktop	new
3619	861	control	2025-09-05 07:17:00	mobile	returning
384	897	treatment	2025-09-02 08:32:00	mobile	new
4407	897	control	2025-09-06 00:23:00	desktop	returning
30629	910	control	2025-09-29 10:07:00	desktop	new
31095	910	treatment	2025-09-29 20:51:00	mobile	returning
24237	1229	treatment	2025-09-23 20:09:00	mobile	new
25463	1229	control	2025-09-24 22:50:00	desktop	returning

In [23]: *# Number of mixed users by their variants*

```
pd.crosstab(  
    mixed_assignments["user_id"],  
    mixed_assignments["variant"]  
).value_counts()
```



```
Out[23]: control  treatment
1          1          284
Name: count, dtype: int64
```

```
In [24]: # Examine reassignment timing
mixed_assignments.sort_values(
    ["user_id", "assigned_at"]
).head(50)
```

Out[24]:

	user_id	variant	assigned_at	device	visitor_type
8324	7	treatment	2025-09-09 12:26:00	mobile	returning
12120	7	control	2025-09-12 22:35:00	desktop	returning
13268	189	treatment	2025-09-13 23:03:00	desktop	new
16256	189	control	2025-09-16 16:56:00	mobile	returning
3250	260	control	2025-09-04 22:53:00	desktop	returning
4951	260	treatment	2025-09-06 12:20:00	mobile	returning
7615	432	treatment	2025-09-08 21:35:00	desktop	returning
7824	432	control	2025-09-09 02:06:00	mobile	returning
16416	522	treatment	2025-09-16 20:08:00	mobile	new
18634	522	control	2025-09-18 20:04:00	desktop	returning
29171	634	control	2025-09-28 04:16:00	mobile	new
31264	634	treatment	2025-09-30 00:23:00	desktop	returning
642	861	treatment	2025-09-02 14:02:00	desktop	new
3619	861	control	2025-09-05 07:17:00	mobile	returning
384	897	treatment	2025-09-02 08:32:00	mobile	new
4407	897	control	2025-09-06 00:23:00	desktop	returning
30629	910	control	2025-09-29 10:07:00	desktop	new
31095	910	treatment	2025-09-29 20:51:00	mobile	returning
24237	1229	treatment	2025-09-23 20:09:00	mobile	new
25463	1229	control	2025-09-24 22:50:00	desktop	returning
22230	1276	control	2025-09-22 01:56:00	desktop	returning
26554	1276	treatment	2025-09-25 21:46:00	mobile	returning
27314	1281	control	2025-09-26 14:15:00	desktop	returning
30911	1281	treatment	2025-09-29 16:28:00	mobile	returning

	user_id	variant	assigned_at	device	visitor_type
22108	1326	control	2025-09-21 22:56:00	desktop	new
22791	1326	treatment	2025-09-22 14:00:00	mobile	returning
25146	1396	control	2025-09-24 15:32:00	desktop	new
28319	1396	treatment	2025-09-27 11:15:00	mobile	returning
4847	1595	control	2025-09-06 10:14:00	mobile	returning
6334	1595	treatment	2025-09-07 18:08:00	desktop	returning
2760	1710	control	2025-09-04 11:52:00	mobile	new
7129	1710	treatment	2025-09-08 11:38:00	desktop	returning
12682	1714	treatment	2025-09-13 11:04:00	desktop	new
17203	1714	control	2025-09-17 13:22:00	mobile	returning
8517	1822	control	2025-09-09 16:45:00	mobile	new
10891	1822	treatment	2025-09-11 20:38:00	desktop	returning
21442	1996	treatment	2025-09-21 08:40:00	desktop	returning
22583	1996	control	2025-09-22 09:33:00	mobile	returning
24979	2392	control	2025-09-24 12:01:00	desktop	new
26804	2392	treatment	2025-09-26 03:22:00	mobile	returning
12471	2496	treatment	2025-09-13 06:32:00	mobile	new
17015	2496	control	2025-09-17 09:01:00	desktop	returning
8617	2597	treatment	2025-09-09 18:50:00	desktop	returning
13745	2597	control	2025-09-14 09:48:00	mobile	returning
24767	2762	control	2025-09-24 07:12:00	mobile	returning
30334	2762	treatment	2025-09-29 04:21:00	desktop	returning
27330	2810	treatment	2025-09-26 14:33:00	desktop	new
28784	2810	control	2025-09-27 20:18:00	mobile	returning

	user_id	variant	assigned_at	device	visitor_type
15608	3123	treatment	2025-09-16 03:11:00	desktop	new
16524	3123	control	2025-09-16 22:40:00	mobile	returning

```
In [25]: # Variant distribution in the raw assignment data
df1["variant"].value_counts()
```

```
Out[25]: variant
treatment    15661
control      15623
Name: count, dtype: int64
```

```
In [26]: # Variant percentages
df1["variant"].value_counts(normalize=True).mul(100).round(2)
```

```
Out[26]: variant
treatment    50.06
control      49.94
Name: proportion, dtype: float64
```

```
In [27]: # Check duplicate user-variant assignments
duplicate_user_variant = (
    df1.groupby(["user_id", "variant"])
        .size()
        .reset_index(name="assignment_count")
)

duplicate_user_variant[
    duplicate_user_variant["assignment_count"] > 1
].head(20)
```

```
Out[27]:  user_id  variant  assignment_count
```

```
In [28]: print(
    "User-variant combinations with multiple assignments:",
    (duplicate_user_variant["assignment_count"] > 1).sum()
)
```

User-variant combinations with multiple assignments: 0

```
In [29]: print("Total assignment rows:", len(df1))
print("Unique users:", df1["user_id"].nunique())
```

Total assignment rows: 31284

Unique users: 31000

```
In [30]: # Distribution of assignment counts per user
assignment_counts = df1.groupby("user_id").size()

assignment_counts.value_counts().sort_index()
```

```
Out[30]: 1    30716
2         284
Name: count, dtype: int64
```

```
In [31]: print("Variants:")
print(df1["variant"].value_counts(dropna=False))

print("\nDevices:")
print(df1["device"].value_counts(dropna=False))

print("\nVisitor Types:")
print(df1["visitor_type"].value_counts(dropna=False))
```

Variants:

variant

treatment 15661

control 15623

Name: count, dtype: int64

Devices:

device

desktop 15282

mobile 13472

tablet 2530

Name: count, dtype: int64

Visitor Types:

visitor_type

new 19300

returning 11984

Name: count, dtype: int64

```
In [32]: print("\nMissing values:")
print(df1.isna().sum())
```

```
Missing values:
user_id      0
variant      0
assigned_at   0
device       0
visitor_type  0
dtype: int64
```

```
In [33]: df2 = pd.read_csv(
    r"C:\Users\papus\OneDrive\Documents\Data Lab Project\Redesign AB Testing\orders.csv"
)

df2.head()
```

```
Out[33]:
```

	order_id	user_id	ordered_at	order_value
0	900001	8321	28-08-2025 18:45	46.93
1	900002	8275	29-08-2025 05:07	34.88
2	900003	11704	31-08-2025 06:14	64.13
3	900004	16712	31-08-2025 10:35	56.39
4	900005	758	31-08-2025 19:23	121.54

```
In [34]: # Dataset shape
df2.shape
```

```
Out[34]: (2670, 4)
```

```
In [35]: # Column names
df2.columns.tolist()
```

```
Out[35]: ['order_id', 'user_id', 'ordered_at', 'order_value']
```

```
In [36]: # Data types and non-null counts
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2670 entries, 0 to 2669
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   order_id    2670 non-null   int64
1   user_id     2670 non-null   int64
2   ordered_at  2670 non-null   object
3   order_value 2670 non-null   float64
dtypes: float64(1), int64(2), object(1)
memory usage: 83.6+ KB
```

```
In [39]: df2["ordered_at"] = pd.to_datetime(
        df2["ordered_at"],
        format="%d-%m-%Y %H:%M",
        errors="coerce"
    )
```

```
In [40]: df2.isna().sum()
```

```
Out[40]: order_id      0
        user_id      0
        ordered_at    0
        order_value    0
        dtype: int64
```

```
In [41]: df2.describe(include="all").T
```

```
Out[41]:
```

	count	mean	min	25%	50%	75%	max	std
order_id	2670.0	901335.5	900001.0	900668.25	901335.5	902002.75	902670.0	770.906933
user_id	2670.0	15482.746816	19.0	7756.75	15529.5	23055.25	30989.0	8888.698398
ordered_at	2670	2025-09-16 04:33:15.797752576	2025-08-28 18:45:00	2025-09-09 01:42:30	2025-09-16 07:28:30	2025-09-23 03:23:45	2025-09-29 23:39:00	NaN
order_value	2670.0	63.973532	9.0	45.5925	64.17	81.8075	147.48	26.016574

```
In [42]: print("Total orders:", len(df2))
        print("Unique order IDs:", df2["order_id"].nunique())

        print(
            "Duplicate order IDs:",
```

```
df2["order_id"].duplicated().sum()  
)
```

Total orders: 2670
Unique order IDs: 2670
Duplicate order IDs: 0

```
In [43]: print("Zero-value orders:", (df2["order_value"] == 0).sum())  
print("Negative-value orders:", (df2["order_value"] < 0).sum())
```

Zero-value orders: 0
Negative-value orders: 0

```
In [44]: print("Missing ordered_at:", df2["ordered_at"].isna().sum())  
print("Earliest order:", df2["ordered_at"].min())  
print("Latest order:", df2["ordered_at"].max())
```

Missing ordered_at: 0
Earliest order: 2025-08-28 18:45:00
Latest order: 2025-09-29 23:39:00

```
In [45]: outside_orders = df2[  
    (df2["ordered_at"] < "2025-09-02") |  
    (df2["ordered_at"] >= "2025-09-30")  
]  
  
print("Orders outside experiment window:", len(outside_orders))
```

Orders outside experiment window: 11

```
In [46]: assigned_user_ids = set(df1["user_id"])  
  
unassigned_orders = df2[  
    ~df2["user_id"].isin(assigned_user_ids)  
]  
  
print("Orders from users not found in assignments:", len(unassigned_orders))
```

Orders from users not found in assignments: 0

```
In [47]: outside_orders = df2[  
    (df2["ordered_at"] < "2025-09-02") |  
    (df2["ordered_at"] >= "2025-09-30")  
]  
  
outside_orders.sort_values("ordered_at")
```


Out[47]:

	order_id	user_id	ordered_at	order_value
0	900001	8321	2025-08-28 18:45:00	46.93
1	900002	8275	2025-08-29 05:07:00	34.88
2	900003	11704	2025-08-31 06:14:00	64.13
3	900004	16712	2025-08-31 10:35:00	56.39
4	900005	758	2025-08-31 19:23:00	121.54
5	900006	14246	2025-09-01 04:28:00	77.35
6	900007	4565	2025-09-01 11:22:00	83.26
7	900008	1411	2025-09-01 12:05:00	62.06
8	900009	22170	2025-09-01 12:52:00	104.60
9	900010	20372	2025-09-01 17:53:00	57.45
10	900011	30964	2025-09-01 22:53:00	50.78

In [48]:

```
outside_orders.merge(  
    df1[["user_id", "variant", "assigned_at"]],  
    on="user_id",  
    how="left"  
).sort_values(["user_id", "ordered_at"])
```

Out[48]:

	order_id	user_id	ordered_at	order_value	variant	assigned_at
5	900005	758	2025-08-31 19:23:00	121.54	treatment	2025-09-03 17:12:00
8	900008	1411	2025-09-01 12:05:00	62.06	control	2025-09-04 18:24:00
7	900007	4565	2025-09-01 11:22:00	83.26	control	2025-09-05 00:16:00
2	900002	8275	2025-08-29 05:07:00	34.88	control	2025-09-03 18:25:00
0	900001	8321	2025-08-28 18:45:00	46.93	control	2025-09-03 04:49:00
1	900001	8321	2025-08-28 18:45:00	46.93	treatment	2025-09-06 14:56:00
3	900003	11704	2025-08-31 06:14:00	64.13	control	2025-09-03 02:41:00
6	900006	14246	2025-09-01 04:28:00	77.35	treatment	2025-09-04 17:01:00
4	900004	16712	2025-08-31 10:35:00	56.39	treatment	2025-09-05 08:31:00
10	900010	20372	2025-09-01 17:53:00	57.45	control	2025-09-03 21:29:00
9	900009	22170	2025-09-01 12:52:00	104.60	treatment	2025-09-06 23:09:00
11	900011	30964	2025-09-01 22:53:00	50.78	control	2025-09-03 18:52:00

```
In [49]: mixed_orders = df2[
          df2["user_id"].isin(mixed_users)
          ].copy()

          print("Orders from mixed-variant users:", len(mixed_orders))
```

Orders from mixed-variant users: 19

```
In [50]: mixed_orders.sort_values(
          ["user_id", "ordered_at"]
          ).head(30)
```

Out[50]:

	order_id	user_id	ordered_at	order_value
2634	902635	3469	2025-09-29 15:24:00	90.15
430	900431	3724	2025-09-06 18:21:00	94.31
1002	901003	5220	2025-09-12 18:09:00	52.51
1869	901870	6303	2025-09-21 22:57:00	82.85
970	900971	6806	2025-09-12 11:04:00	36.60
2432	902433	7332	2025-09-27 11:53:00	53.09
2517	902518	7332	2025-09-28 12:31:00	78.89
0	900001	8321	2025-08-28 18:45:00	46.93
2387	902388	8409	2025-09-27 02:02:00	94.23
2303	902304	9662	2025-09-26 08:21:00	76.56
798	900799	13645	2025-09-10 11:00:00	61.28
1269	901270	14517	2025-09-15 15:39:00	55.50
324	900325	15803	2025-09-05 16:33:00	44.12
303	900304	15896	2025-09-05 12:24:00	88.92
586	900587	15896	2025-09-08 06:49:00	101.90
436	900437	21178	2025-09-06 18:55:00	97.12
2394	902395	26442	2025-09-27 02:41:00	19.27
1289	901290	26599	2025-09-15 20:55:00	51.95
64	900065	29350	2025-09-02 19:31:00	84.17

In [51]: *# Get assignment history for mixed-variant users*

```
mixed_assignments = (  
    df1[df1["user_id"].isin(mixed_users)]  
    [ ["user_id", "variant", "assigned_at"]  
      .sort_values(["user_id", "assigned_at"])  
    ]  
)
```

```
# Merge orders with assignment history
mixed_order_timing = mixed_orders.merge(
    mixed_assignments,
    on="user_id",
    how="left"
)

# Calculate timing relative to each assignment
mixed_order_timing["order_after_assignment"] = (
    mixed_order_timing["ordered_at"] >
    mixed_order_timing["assigned_at"]
)

mixed_order_timing.sort_values(
    ["user_id", "ordered_at", "assigned_at"]
)
```

Out[51]:

	order_id	user_id	ordered_at	order_value	variant	assigned_at	order_after_assignment
36	902635	3469	2025-09-29 15:24:00	90.15	treatment	2025-09-29 05:13:00	True
37	902635	3469	2025-09-29 15:24:00	90.15	control	2025-09-30 13:39:00	False
8	900431	3724	2025-09-06 18:21:00	94.31	control	2025-09-06 03:35:00	True
9	900431	3724	2025-09-06 18:21:00	94.31	treatment	2025-09-11 02:01:00	False
18	901003	5220	2025-09-12 18:09:00	52.51	control	2025-09-12 07:02:00	True
19	901003	5220	2025-09-12 18:09:00	52.51	treatment	2025-09-16 23:06:00	False
24	901870	6303	2025-09-21 22:57:00	82.85	treatment	2025-09-21 08:12:00	True
25	901870	6303	2025-09-21 22:57:00	82.85	control	2025-09-21 17:38:00	True
16	900971	6806	2025-09-12 11:04:00	36.60	treatment	2025-09-12 07:51:00	True
17	900971	6806	2025-09-12 11:04:00	36.60	control	2025-09-12 23:43:00	False
32	902433	7332	2025-09-27 11:53:00	53.09	treatment	2025-09-27 05:57:00	True
33	902433	7332	2025-09-27 11:53:00	53.09	control	2025-09-28 15:09:00	False
34	902518	7332	2025-09-28 12:31:00	78.89	treatment	2025-09-27 05:57:00	True
35	902518	7332	2025-09-28 12:31:00	78.89	control	2025-09-28 15:09:00	False
0	900001	8321	2025-08-28 18:45:00	46.93	control	2025-09-03 04:49:00	False
1	900001	8321	2025-08-28 18:45:00	46.93	treatment	2025-09-06 14:56:00	False
28	902388	8409	2025-09-27 02:02:00	94.23	control	2025-09-26 22:35:00	True
29	902388	8409	2025-09-27 02:02:00	94.23	treatment	2025-09-29 17:17:00	False
26	902304	9662	2025-09-26 08:21:00	76.56	treatment	2025-09-26 05:55:00	True
27	902304	9662	2025-09-26 08:21:00	76.56	control	2025-09-30 20:32:00	False
14	900799	13645	2025-09-10 11:00:00	61.28	treatment	2025-09-09 12:11:00	True
15	900799	13645	2025-09-10 11:00:00	61.28	control	2025-09-10 09:05:00	True
20	901270	14517	2025-09-15 15:39:00	55.50	control	2025-09-15 15:11:00	True
21	901270	14517	2025-09-15 15:39:00	55.50	treatment	2025-09-17 01:30:00	False

	order_id	user_id	ordered_at	order_value	variant	assigned_at	order_after_assignment
6	900325	15803	2025-09-05 16:33:00	44.12	treatment	2025-09-05 03:37:00	True
7	900325	15803	2025-09-05 16:33:00	44.12	control	2025-09-06 03:30:00	False
4	900304	15896	2025-09-05 12:24:00	88.92	treatment	2025-09-05 00:16:00	True
5	900304	15896	2025-09-05 12:24:00	88.92	control	2025-09-06 14:43:00	False
12	900587	15896	2025-09-08 06:49:00	101.90	treatment	2025-09-05 00:16:00	True
13	900587	15896	2025-09-08 06:49:00	101.90	control	2025-09-06 14:43:00	True
10	900437	21178	2025-09-06 18:55:00	97.12	control	2025-09-06 17:30:00	True
11	900437	21178	2025-09-06 18:55:00	97.12	treatment	2025-09-10 07:13:00	False
30	902395	26442	2025-09-27 02:41:00	19.27	treatment	2025-09-26 02:08:00	True
31	902395	26442	2025-09-27 02:41:00	19.27	control	2025-09-29 07:28:00	False
22	901290	26599	2025-09-15 20:55:00	51.95	treatment	2025-09-15 19:46:00	True
23	901290	26599	2025-09-15 20:55:00	51.95	control	2025-09-17 22:53:00	False
2	900065	29350	2025-09-02 19:31:00	84.17	treatment	2025-09-02 15:33:00	True
3	900065	29350	2025-09-02 19:31:00	84.17	control	2025-09-07 05:11:00	False

```
In [52]: mixed_order_timing.groupby(
          ["user_id", "ordered_at", "order_id"]
        )["order_after_assignment"].sum().value_counts()
```

```
Out[52]: order_after_assignment
1      15
2       3
0       1
Name: count, dtype: int64
```

```
In [53]: # Orders from mixed-variant users
mixed_order_ids = set(
    df2.loc[df2["user_id"].isin(mixed_users), "order_id"]
)

# Orders before the user's first assignment
```

```

first_assignment = (
    df1.groupby("user_id")["assigned_at"]
        .min()
        .rename("first_assigned_at")
)

orders_with_assignment = df2.merge(
    first_assignment,
    on="user_id",
    how="left"
)

pre_assignment_orders = orders_with_assignment[
    orders_with_assignment["ordered_at"] <
    orders_with_assignment["first_assigned_at"]
]

pre_assignment_order_ids = set(
    pre_assignment_orders["order_id"]
)

print("Orders from mixed users:", len(mixed_order_ids))
print("Pre-assignment orders:", len(pre_assignment_order_ids))
print(
    "Overlap between the two:",
    len(mixed_order_ids & pre_assignment_order_ids)
)

```

Orders from mixed users: 19

Pre-assignment orders: 86

Overlap between the two: 1

```

In [54]: # Unique orders excluded
orders_to_remove = mixed_order_ids | pre_assignment_order_ids

print("Total unique orders to exclude:", len(orders_to_remove))

```

Total unique orders to exclude: 104

```

In [55]: # Users assigned to both control and treatment
mixed_user_ids = set(mixed_users)

print("Users to exclude:", len(mixed_user_ids))

```

Users to exclude: 284

```
In [56]: # Remove mixed-variant users
df1_clean = df1[
    ~df1["user_id"].isin(mixed_user_ids)
].copy()

print("Original assignment rows:", len(df1))
print("Clean assignment rows:", len(df1_clean))
print("Unique users in clean data:", df1_clean["user_id"].nunique())
```

Original assignment rows: 31284
Clean assignment rows: 30716
Unique users in clean data: 30716

```
In [57]: print(
    df1_clean.groupby("variant")["user_id"]
    .nunique()
)
```

variant
control 15339
treatment 15377
Name: user_id, dtype: int64

```
In [58]: print(
    df1_clean["variant"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)
```

variant
treatment 50.06
control 49.94
Name: proportion, dtype: float64

```
In [59]: # Keep only orders from clean experiment users
valid_orders = df2[
    df2["user_id"].isin(df1_clean["user_id"])
].copy()

print("Orders from clean experiment users:", len(valid_orders))
```

Orders from clean experiment users: 2651


```
In [60]: valid_orders = valid_orders.merge(
    df1_clean[
        ["user_id", "variant", "assigned_at"]
    ],
    on="user_id",
    how="left"
)

valid_orders.head()
```

```
Out[60]:
```

	order_id	user_id	ordered_at	order_value	variant	assigned_at
0	900002	8275	2025-08-29 05:07:00	34.88	control	2025-09-03 18:25:00
1	900003	11704	2025-08-31 06:14:00	64.13	control	2025-09-03 02:41:00
2	900004	16712	2025-08-31 10:35:00	56.39	treatment	2025-09-05 08:31:00
3	900005	758	2025-08-31 19:23:00	121.54	treatment	2025-09-03 17:12:00
4	900006	14246	2025-09-01 04:28:00	77.35	treatment	2025-09-04 17:01:00

```
In [61]: valid_orders = valid_orders[
    valid_orders["ordered_at"] > valid_orders["assigned_at"]
].copy()

print("Valid post-assignment orders:", len(valid_orders))
```

Valid post-assignment orders: 2560

```
In [62]: print("Orders by variant:")
print(valid_orders["variant"].value_counts())

print("\nUsers with valid orders:")
print(valid_orders.groupby("variant")["user_id"].nunique())
```

```
Orders by variant:
variant
treatment    1320
control       1240
Name: count, dtype: int64
```

```
Users with valid orders:
variant
control       1113
treatment     1185
Name: user_id, dtype: int64
```

Conversion Rate

```
In [63]: # Users with at least one valid order
converted_users = (
    valid_orders.groupby("variant")["user_id"]
    .nunique()
)

# Total assigned users
total_users = (
    df1_clean.groupby("variant")["user_id"]
    .nunique()
)

# Conversion rate
conversion_rate = (
    converted_users / total_users * 100
).round(2)

conversion_rate
```

```
Out[63]: variant
control    7.26
treatment  7.71
Name: user_id, dtype: float64
```

```
In [64]: conversion_summary = pd.DataFrame({
    "Users": total_users,
    "Converted Users": converted_users,
    "Conversion Rate (%)": conversion_rate
})
```

conversion_summary

Out[64]:

	Users	Converted Users	Conversion Rate (%)
--	-------	-----------------	---------------------

variant			
control	15339	1113	7.26
treatment	15377	1185	7.71

conversion lift

```
In [65]: control_rate = converted_users["control"] / total_users["control"]
treatment_rate = converted_users["treatment"] / total_users["treatment"]

absolute_difference = treatment_rate - control_rate
relative_lift = absolute_difference / control_rate

print(f"Absolute difference: {absolute_difference:.4%}")
print(f"Relative lift: {relative_lift:.2%}")
```

Absolute difference: 0.4503%

Relative lift: 6.21%

Two-proportion z-test

```
In [66]: from statsmodels.stats.proportion import proportions_ztest

counts = np.array([
    converted_users["treatment"],
    converted_users["control"]
])

nobs = np.array([
    total_users["treatment"],
    total_users["control"]
])

z_stat, p_value = proportions_ztest(
    count=counts,
    nobs=nobs,
```

```

        alternative="two-sided"
    )

    print(f"Z-statistic: {z_stat:.4f}")
    print(f"P-value: {p_value:.4f}")

```

Z-statistic: 1.4998

P-value: 0.1337

In [67]: `from statsmodels.stats.proportion import proportion_confint`

```

# Conversion rates
p_treatment = converted_users["treatment"] / total_users["treatment"]
p_control = converted_users["control"] / total_users["control"]

# Difference: Treatment - Control
diff = p_treatment - p_control

# Standard error for independent proportions
se = np.sqrt(
    (p_treatment * (1 - p_treatment) / total_users["treatment"]) +
    (p_control * (1 - p_control) / total_users["control"])
)

# 95% Confidence Interval
z_critical = norm.ppf(0.975)

ci_lower = diff - z_critical * se
ci_upper = diff + z_critical * se

print(f"Difference: {diff:.4%}")
print(f"95% CI Lower: {ci_lower:.4%}")
print(f"95% CI Upper: {ci_upper:.4%}")

```

Difference: 0.4503%

95% CI Lower: -0.1381%

95% CI Upper: 1.0387%

AOV

In [68]: `# AOV by variant`

```

aov = (
    valid_orders.groupby("variant")
    .agg(

```

```

        total_revenue=("order_value", "sum"),
        total_orders=("order_id", "nunique")
    )
)

aov["AOV"] = aov["total_revenue"] / aov["total_orders"]

print(aov)

```

	total_revenue	total_orders	AOV
variant			
control	78740.25	1240	63.500202
treatment	85232.21	1320	64.569856

Revenue per User

```

In [69]: # Revenue per assigned user
rpu = (
    valid_orders.groupby("variant")["order_value"]
    .sum()
    .div(total_users)
)

print("\nRevenue per User:")
print(rpu)

```

Revenue per User:

variant	
control	5.133337
treatment	5.542837

dtype: float64

Comparing treatment vs control

```

In [70]: aov_lift = (
    (aov.loc["treatment", "AOV"] /
     aov.loc["control", "AOV"]) - 1
) * 100

rpu_lift = (
    (rpu["treatment"] /
     rpu["control"]) - 1
) * 100

```

```
print(f"AOV lift: {aov_lift:.2f}%")
print(f"RPU lift: {rpu_lift:.2f}%")
```

AOV lift: 1.68%
RPU lift: 7.98%

Experiment Results — Primary & Secondary Metrics

Experiment Results — Primary & Secondary Metrics

The primary objective of the experiment is to determine whether the redesigned checkout increased the proportion of assigned users who placed at least one order after assignment.

Two secondary business metrics were also evaluated:

- **Average Order Value (AOV):** Average revenue generated per completed order.
- **Revenue per User (RPU):** Total experiment revenue divided by the total number of assigned users.

All metrics are calculated using the cleaned experiment population after excluding users exposed to both variants and orders occurring before user assignment.

1 Primary Metric — Conversion Rate

Conversion is defined at the user level:

A user is considered converted if they placed at least one valid order after their experiment assignment.

A user with multiple orders is counted only once for the primary conversion metric.

Metric	Control	Treatment
Assigned users	15,339	15,377
Converted users	1,113	1,185
Conversion rate	7.26%	7.71%
Absolute difference	—	+0.45 pp

Metric	Control	Treatment
Relative lift	—	+6.21%

The treatment group had a conversion rate of 7.71%, compared with 7.26% for the control group. This represents an observed absolute increase of 0.45 percentage points and a relative lift of 6.21%.

2 Statistical Significance — Two-Proportion Z-Test

Because the primary metric is a binary conversion outcome, a two-proportion z-test was used to compare the conversion rates between the treatment and control groups.

Null hypothesis (H_0):

The treatment and control groups have the same conversion rate.

Alternative hypothesis (H_1):

The treatment and control groups have different conversion rates.

The significance level was set at $\alpha = 0.05$.

Test Results

- Z-statistic: **1.4998**
- P-value: **0.1337**
- Significance level: **0.05**

Since the p-value (0.1337) is greater than 0.05, the observed difference in conversion rates is **not statistically significant at the 5% level**.

Therefore, the experiment does not provide sufficient statistical evidence to conclude that the redesigned checkout changed the primary conversion rate.

95% Confidence Interval

The 95% confidence interval was calculated for the difference in conversion rates between treatment and control.

Treatment – Control

- Observed difference: **+0.4503 percentage points**

- 95% CI: **-0.1381 to +1.0387 percentage points**

The confidence interval includes zero, which is consistent with the two-proportion z-test result.

The observed treatment effect is positive, but the plausible range includes both a small negative effect and a positive effect. Therefore, the experiment does not provide statistically significant evidence of a conversion-rate improvement at the 5% level.

Secondary Metric — Average Order Value (AOV)

AOV is calculated as:

$$\text{AOV} = \text{Total Revenue} / \text{Total Orders}$$

Metric	Control	Treatment	Change
Total revenue	\$78,740.25	\$85,232.21	+8.25%
Total orders	1,240	1,320	+6.45%
AOV	\$63.50	\$64.57	+1.68%

The treatment group generated an observed AOV of 64.57*compared with* 63.50 for the control group, representing a 1.68% increase.

This is an observed difference; statistical significance for AOV has not been tested at this stage.

Secondary Metric — Revenue per User (RPU)

Revenue per User is calculated using all assigned users in each variant:

$$\text{RPU} = \text{Total Valid Revenue} / \text{Total Assigned Users}$$

Metric	Control	Treatment	Change
Total revenue	\$78,740.25	\$85,232.21	+8.25%
Assigned users	15,339	15,377	+0.25%
Revenue per User	\$5.13	\$5.54	+7.98%

The treatment group generated 5.54*in revenue per assigned user compared with* 5.13 for the control group, an observed increase of 7.98%.

As with AOV, this is an observed difference and has not yet been subjected to a statistical significance test.

Summary of Findings

The redesigned checkout produced positive observed changes across the primary and secondary business metrics:

- Conversion rate increased from **7.26% to 7.71%** (+0.45 pp).
- AOV increased from 63.50to**64.57** (+1.68%).
- Revenue per User increased from 5.13to**5.54** (+7.98%).

However, the primary conversion-rate difference was **not statistically significant** ($p = 0.1337$), and the 95% confidence interval for the treatment effect ranged from **-0.1381 pp to +1.0387 pp**.

Therefore, the observed positive movement should be treated as directional rather than conclusive evidence of a checkout conversion improvement.

Segment Analysis

```
In [72]: user_attributes = (
          df1_clean[
              ["user_id", "device", "visitor_type"]
          ]
          .drop_duplicates("user_id")
          )

valid_orders = valid_orders.merge(
    user_attributes,
    on="user_id",
    how="left"
)

print(valid_orders[["user_id", "device", "visitor_type"]].head())
print("\nMissing device:", valid_orders["device"].isna().sum())
print("Missing visitor type:", valid_orders["visitor_type"].isna().sum())
```

	user_id	device	visitor_type
0	14953	mobile	returning
1	23844	desktop	new
2	30404	mobile	returning
3	16308	tablet	returning
4	80	desktop	returning

Missing device: 0

Missing visitor type: 0

```
In [73]: # Total assigned users by device and variant
device_users = (
    df1_clean
    .groupby(["device", "variant"])["user_id"]
    .nunique()
    .unstack(fill_value=0)
)

# Converted users by device and variant
device_converted = (
    valid_orders
    .groupby(["device", "variant"])["user_id"]
    .nunique()
    .unstack(fill_value=0)
)

# Conversion rate
device_conversion_rate = (
    device_converted
    .div(device_users)
    * 100
)

print("Assigned Users:")
print(device_users)

print("\nConverted Users:")
print(device_converted)

print("\nDevice Conversion Rate (%):")
print(device_conversion_rate.round(2))
```

Assigned Users:

variant	control	treatment
desktop	7410	7588
mobile	6658	6553
tablet	1271	1236

Converted Users:

variant	control	treatment
desktop	558	767
mobile	471	355
tablet	84	63

Device Conversion Rate (%):

variant	control	treatment
desktop	7.53	10.11
mobile	7.07	5.42
tablet	6.61	5.10

```
In [74]: device_effect = pd.DataFrame({
    "control_conversion": device_conversion_rate["control"],
    "treatment_conversion": device_conversion_rate["treatment"]
})

device_effect["absolute_difference_pp"] = (
    device_effect["treatment_conversion"]
    - device_effect["control_conversion"]
)

device_effect["relative_lift_pct"] = (
    (device_effect["treatment_conversion"]
     / device_effect["control_conversion"]) - 1
) * 100

print(device_effect.round(2))
```

	control_conversion	treatment_conversion	absolute_difference_pp \
device			
desktop	7.53	10.11	2.58
mobile	7.07	5.42	-1.66
tablet	6.61	5.10	-1.51

	relative_lift_pct
device	
desktop	34.23
mobile	-23.42
tablet	-22.88

```
In [75]: device_test_results = []

for device in device_users.index:

    control_converted = device_converted.loc[device, "control"]
    treatment_converted = device_converted.loc[device, "treatment"]

    control_users = device_users.loc[device, "control"]
    treatment_users = device_users.loc[device, "treatment"]

    counts = np.array([
        treatment_converted,
        control_converted
    ])

    nobs = np.array([
        treatment_users,
        control_users
    ])

    z_stat, p_value = proportions_ztest(
        count=counts,
        nobs=nobs,
        alternative="two-sided"
    )

    device_test_results.append({
        "device": device,
        "z_stat": z_stat,
        "p_value": p_value
    })
```

```
device_test_results = pd.DataFrame(device_test_results)
```

```
print(device_test_results.round(4))
```

	device	z_stat	p_value
0	desktop	5.5614	0.0000
1	mobile	-3.9328	0.0001
2	tablet	-1.6109	0.1072

```
In [76]: from statsmodels.stats.multitest import multipletests
```

```
device_test_results["adjusted_p_value"] = multipletests(  
    device_test_results["p_value"],  
    method="fdr_bh"  
)[1]
```

```
device_test_results["significant_after_correction"] = (  
    device_test_results["adjusted_p_value"] < 0.05  
)
```

```
print(device_test_results.round(4))
```

	device	z_stat	p_value	adjusted_p_value	significant_after_correction
0	desktop	5.5614	0.0000	0.0000	True
1	mobile	-3.9328	0.0001	0.0001	True
2	tablet	-1.6109	0.1072	0.1072	False

```
In [77]: # Total assigned users by visitor type and variant
```

```
visitor_users = (  
    df1_clean  
    .groupby(["visitor_type", "variant"])["user_id"]  
    .nunique()  
    .unstack(fill_value=0)  
)
```

```
# Converted users by visitor type and variant
```

```
visitor_converted = (  
    valid_orders  
    .groupby(["visitor_type", "variant"])["user_id"]  
    .nunique()  
    .unstack(fill_value=0)  
)
```

```
# Conversion rate
```

```
visitor_conversion_rate = (  
    visitor_converted  
    .div(visitor_users, axis=1)  
    .unstack(fill_value=0)  
)
```

```

        visitor_converted
        .div(visitor_users)
        * 100
    )

    print("Assigned Users:")
    print(visitor_users)

    print("\nConverted Users:")
    print(visitor_converted)

    print("\nVisitor Type Conversion Rate (%):")
    print(visitor_conversion_rate.round(2))

```

Assigned Users:

variant	control	treatment
visitor_type		
new	9513	9611
returning	5826	5766

Converted Users:

variant	control	treatment
visitor_type		
new	580	596
returning	533	589

Visitor Type Conversion Rate (%):

variant	control	treatment
visitor_type		
new	6.10	6.20
returning	9.15	10.22

In [78]: visitor_test_results = []

```

for visitor_type in visitor_users.index:

    control_converted = visitor_converted.loc[visitor_type, "control"]
    treatment_converted = visitor_converted.loc[visitor_type, "treatment"]

    control_users = visitor_users.loc[visitor_type, "control"]
    treatment_users = visitor_users.loc[visitor_type, "treatment"]

    counts = np.array([
        treatment_converted,

```

```

        control_converted
    ])

    nobs = np.array([
        treatment_users,
        control_users
    ])

    z_stat, p_value = proportions_ztest(
        count=counts,
        nobs=nobs,
        alternative="two-sided"
    )

    visitor_test_results.append({
        "visitor_type": visitor_type,
        "z_stat": z_stat,
        "p_value": p_value
    })

visitor_test_results = pd.DataFrame(visitor_test_results)

print(visitor_test_results.round(4))

  visitor_type  z_stat  p_value
0         new    0.3002   0.7640
1    returning    1.9416   0.0522

```

```

In [79]: visitor_test_results["adjusted_p_value"] = multipletests(
        visitor_test_results["p_value"],
        method="fdr_bh"
    )[1]

    visitor_test_results["significant_after_correction"] = (
        visitor_test_results["adjusted_p_value"] < 0.05
    )

    print(visitor_test_results.round(4))

```

```

visitor_type  z_stat  p_value  adjusted_p_value  \
0           new  0.3002   0.7640           0.7640
1    returning  1.9416   0.0522           0.1044

significant_after_correction
0                          False
1                          False

```

Segment Analysis

To investigate whether the treatment effect varied across important user groups, conversion rates were analyzed by:

- Device type: Desktop, Mobile, Tablet
- Visitor type: New, Returning

Two-sided two-proportion z-tests were performed within each segment. Because multiple segment comparisons were performed, Benjamini-Hochberg false discovery rate correction was applied to the resulting p-values.

Conversion by Device

Device	Control	Treatment	Difference	Relative Lift	Adjusted p-value	Significant?
Desktop	7.53%	10.11%	+2.58 pp	+34.23%	<0.0001	Yes
Mobile	7.07%	5.42%	-1.66 pp	-23.42%	0.0001	Yes
Tablet	6.61%	5.10%	-1.51 pp	-22.88%	0.1072	No

The treatment showed a statistically significant increase in conversion among desktop users and a statistically significant decrease among mobile users after multiple-testing correction. The tablet segment showed a negative observed difference, but it was not statistically significant.

The opposite direction of the desktop and mobile effects is an important diagnostic finding and warrants further investigation before interpreting the overall experiment result.

Conversion by Visitor Type

Visitor Type	Control	Treatment	Difference	Adjusted p-value	Significant?
New	6.10%	6.20%	+0.10 pp	0.7640	No

Visitor Type	Control	Treatment	Difference	Adjusted p-value	Significant?
Returning	9.15%	10.22%	+1.07 pp	0.1044	No

Treatment conversion was slightly higher for both new and returning users. However, neither difference was statistically significant after multiple-testing correction.

Segment Analysis Summary

The segment analysis shows that the treatment effect is not uniform across devices:

- Desktop users showed a +2.58 percentage-point observed increase in conversion.
- Mobile users showed a -1.66 percentage-point observed decrease.
- Tablet users showed a -1.51 percentage-point observed decrease, although this was not statistically significant.
- Neither new nor returning users showed a statistically significant treatment effect after correction.

These results should be interpreted as segment-level evidence rather than proof of a treatment-by-segment interaction. A formal interaction test would be required to establish whether the treatment effect differs statistically between segments.

Monthly Revenue Impact.

```
In [80]: control_rpu = rpu["control"]
         treatment_rpu = rpu["treatment"]

         incremental_rpu = treatment_rpu - control_rpu

         print(f"Control RPU: ${control_rpu:.4f}")
         print(f"Treatment RPU: ${treatment_rpu:.4f}")
         print(f"Incremental RPU: ${incremental_rpu:.4f}")
```

```
Control RPU: $5.1333
Treatment RPU: $5.5428
Incremental RPU: $0.4095
```

```
In [81]: monthly_users = total_users.sum()

         estimated_incremental_revenue = incremental_rpu * monthly_users
```

```
print(f"Monthly users: {monthly_users:,}")
print(f"Estimated incremental monthly revenue: ${estimated_incremental_revenue:,.2f}")
```

Monthly users: 30,716

Estimated incremental monthly revenue: \$12,578.23

```
In [82]: # Total revenue per user
user_revenue = (
    valid_orders
    .groupby(["user_id", "variant"])["order_value"]
    .sum()
    .reset_index(name="revenue")
)

# Add users with no orders and assign them $0 revenue
all_users = df1_clean[["user_id", "variant"]].copy()

user_revenue = (
    all_users
    .merge(
        user_revenue,
        on=["user_id", "variant"],
        how="left"
    )
)

user_revenue["revenue"] = user_revenue["revenue"].fillna(0)

print(user_revenue.groupby("variant")["revenue"].agg(
    ["count", "mean", "std"]
))
```

	count	mean	std
variant			
control	15339	5.133337	20.543575
treatment	15377	5.542837	21.628235

```
In [83]: # Separate treatment and control revenue
control_revenue = user_revenue.loc[
    user_revenue["variant"] == "control", "revenue"
]

treatment_revenue = user_revenue.loc[
    user_revenue["variant"] == "treatment", "revenue"
]
```

```

# Welch's t-test
t_stat, p_value_rpu = stats.ttest_ind(
    treatment_revenue,
    control_revenue,
    equal_var=False
)

# Difference in mean revenue per user
rpu_difference = (
    treatment_revenue.mean() - control_revenue.mean()
)

# Standard error of the difference
se_rpu = np.sqrt(
    treatment_revenue.var(ddof=1) / len(treatment_revenue)
    +
    control_revenue.var(ddof=1) / len(control_revenue)
)

# Welch-Satterthwaite degrees of freedom
df_welch = (
    (
        treatment_revenue.var(ddof=1) / len(treatment_revenue)
        +
        control_revenue.var(ddof=1) / len(control_revenue)
    ) ** 2
    /
    (
        (
            treatment_revenue.var(ddof=1) / len(treatment_revenue)
        ) ** 2 / (len(treatment_revenue) - 1)
        +
        (
            control_revenue.var(ddof=1) / len(control_revenue)
        ) ** 2 / (len(control_revenue) - 1)
    )
)

# 95% confidence interval
t_critical = stats.t.ppf(0.975, df_welch)

ci_lower_rpu = rpu_difference - t_critical * se_rpu
ci_upper_rpu = rpu_difference + t_critical * se_rpu

```

```
print(f"RPU difference: ${rpu_difference:.4f}")
print(f"95% CI Lower: ${ci_lower_rpu:.4f}")
print(f"95% CI Upper: ${ci_upper_rpu:.4f}")
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value_rpu:.4f}")
```

```
RPU difference: $0.4095
95% CI Lower: $-0.0623
95% CI Upper: $0.8813
T-statistic: 1.7013
P-value: 0.0889
```

```
In [84]: monthly_users = total_users.sum()

lower_revenue_impact = ci_lower_rpu * monthly_users
point_revenue_impact = rpu_difference * monthly_users
upper_revenue_impact = ci_upper_rpu * monthly_users

print(f"Lower monthly revenue impact: ${lower_revenue_impact:,.2f}")
print(f"Point estimate: ${point_revenue_impact:,.2f}")
print(f"Upper monthly revenue impact: ${upper_revenue_impact:,.2f}")
```

```
Lower monthly revenue impact: $-1,912.83
Point estimate: $12,578.23
Upper monthly revenue impact: $27,069.28
```

Monthly Revenue Impact

Revenue impact was estimated using Revenue per User (RPU), which captures the combined effect of conversion and order value.

The observed RPU difference between treatment and control was:

- Control RPU: **\$5.13**
- Treatment RPU: **\$5.54**
- Incremental RPU: **+\$0.41 per user**

A Welch's two-sample t-test was used to estimate the uncertainty around the RPU difference.

RPU Statistical Results

Metric		Result
RPU difference		+\$0.4095
95% CI	0.0623 to +	– 0.8813
t-statistic		1.7013
p-value		0.0889

The RPU difference was not statistically significant at the 5% level. The confidence interval includes zero, meaning the observed revenue-per-user improvement is uncertain.

Estimated Monthly Revenue Impact

Using the 30,716 eligible experiment users as the monthly baseline:

Scenario	Monthly Revenue Impact
Lower bound	–\$1,912.83
Point estimate	+\$12,578.23
Upper bound	+\$27,069.28

The observed point estimate suggests approximately **\$12.6K in additional monthly revenue**, with a 95% confidence interval ranging from approximately *1.9K monthly downside* to **27.1K monthly upside**.

Because the confidence interval includes zero and the RPU difference is not statistically significant, the revenue impact should be treated as an estimate rather than a guaranteed financial outcome.

Revenue Impact Interpretation

The experiment provides directional evidence of higher revenue per user under the redesigned checkout, but the uncertainty around the estimate is substantial. A longer experiment or larger sample could provide more precise evidence about the underlying revenue effect.

Executive Summary

Checkout Redesign A/B Test — Executive Findings

A four-week A/B test was conducted to evaluate a redesigned checkout experience against the existing checkout. After experiment-integrity checks, 284 users who were exposed to both variants were excluded from the primary analysis, leaving **30,716 eligible users** with an approximately balanced 50/50 treatment split.

Primary Metric

The primary metric was the percentage of assigned users who placed at least one order after assignment.

Metric	Control	Treatment	Treatment Effect
Users	15,339	15,377	—
Converted users	1,113	1,185	—
Conversion rate	7.26%	7.71%	+0.45 pp
Relative lift	—	—	+6.21%
P-value	—	—	0.1337
95% CI	—	—	−0.14 pp to +1.04 pp

The treatment group had a higher observed conversion rate, but the difference was **not statistically significant at the 5% level**.

Secondary Metrics

Metric	Control	Treatment	Observed Change
AOV	\$63.50	\$64.57	+1.68%
Revenue per User	\$5.13	\$5.54	+7.98%

The treatment showed positive observed movement in both AOV and RPU. However, the RPU difference was also not statistically significant (p = 0.0889).

Segment Findings

Device

The treatment effect varied considerably by device:

Device	Control	Treatment	Difference	Adjusted p-value
Desktop	7.53%	10.11%	+2.58 pp	<0.0001
Mobile	7.07%	5.42%	−1.66 pp	0.0001
Tablet	6.61%	5.10%	−1.51 pp	0.1072

Desktop and mobile showed statistically significant treatment-versus-control differences after Benjamini-Hochberg correction, but in opposite directions.

This contrast is an important diagnostic finding. A formal treatment-by-device interaction test would be needed to establish whether the treatment effect itself differs statistically between devices.

Visitor Type

Visitor Type	Control	Treatment	Difference	Adjusted p-value
New	6.10%	6.20%	+0.10 pp	0.7640
Returning	9.15%	10.22%	+1.07 pp	0.1044

Neither visitor-type segment showed a statistically significant treatment effect after multiple-testing correction.

Revenue Impact

Based on the observed RPU difference and the experiment's 30,716 eligible users as the monthly baseline:

Scenario	Estimated Monthly Revenue Impact
Lower bound	−\$1,912.83
Point estimate	+\$12,578.23
Upper bound	+\$27,069.28

The point estimate suggests approximately **+\$12.6K in monthly revenue**, but the confidence interval includes both downside and upside outcomes.

Overall Evidence

The redesigned checkout produced positive observed changes in overall conversion, AOV, and RPU. However:

- The primary conversion result was **not statistically significant**.
- The RPU result was **not statistically significant**.
- The revenue-impact range includes a potential downside.
- Device-level analysis revealed materially different observed effects for desktop and mobile users.
- Visitor-type analysis did not identify statistically significant differences.

The strongest follow-up signal is therefore the **opposite-direction device pattern**, particularly the contrast between desktop and mobile users.

Treatment × Device Interaction Test

```
In [85]: import statsmodels.formula.api as smf

# Create user-level experiment dataset
user_level = df1_clean[
    ["user_id", "variant", "device", "visitor_type"]
].copy()

# Users who converted
converted_user_ids = set(valid_orders["user_id"])

user_level["converted"] = (
    user_level["user_id"]
    .isin(converted_user_ids)
    .astype(int)
)

print(user_level.head())
print("\nConversion counts:")
print(
    user_level.groupby(["device", "variant"])["converted"]
    .agg(["sum", "count"])
)
```


	user_id	variant	device	visitor_type	converted
0	9371	treatment	desktop	new	1
1	14953	control	mobile	returning	1
2	7085	control	desktop	new	0
3	1063	control	mobile	new	0
4	27723	treatment	mobile	new	0

Conversion counts:

		sum	count
device	variant		
desktop	control	558	7410
	treatment	767	7588
mobile	control	471	6658
	treatment	355	6553
tablet	control	84	1271
	treatment	63	1236

```
In [86]: # Logistic regression with treatment x device interaction
interaction_model = smf.logit(
    "converted ~ C(variant) * C(device)",
    data=user_level
).fit()

print(interaction_model.summary())
```

Optimization terminated successfully.
Current function value: 0.263843
Iterations 7

Logit Regression Results

=====						
Dep. Variable:	converted	No. Observations:	30716			
Model:	Logit	Df Residuals:	30710			
Method:	MLE	Df Model:	5			
Date:	Mon, 21 Sep 2026	Pseudo R-squ.:	0.007801			
Time:	20:57:39	Log-Likelihood:	-8104.2			
converged:	True	LL-Null:	-8167.9			
Covariance Type:	nonrobust	LLR p-value:	8.298e-26			
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	-2.5079	0.044	-56.968	0.000	-2.594	-2.422
C(variant)[T.treatment]	0.3227	0.058	5.543	0.000	0.209	0.437
C(device)[T.mobile]	-0.0674	0.065	-1.037	0.300	-0.195	0.060
C(device)[T.tablet]	-0.1404	0.121	-1.159	0.247	-0.378	0.097
C(variant)[T.treatment]:C(device)[T.mobile]	-0.6072	0.093	-6.528	0.000	-0.789	-0.425
C(variant)[T.treatment]:C(device)[T.tablet]	-0.5985	0.181	-3.301	0.001	-0.954	-0.243
=====						

Treatment × Device Interaction

Separate segment-level tests showed statistically significant treatment-versus-control differences for desktop and mobile users. However, separate significance tests do not directly establish whether the treatment effect differs between devices.

To formally test treatment-effect heterogeneity, a logistic regression model with a treatment × device interaction was fitted:

Conversion ~ Variant × Device

Desktop was used as the reference device.

Interaction Results

Interaction Term	Coefficient	P-value
Treatment × Mobile	-0.6072	<0.001
Treatment × Tablet	-0.5985	0.001

Both interaction terms were statistically significant.

This provides evidence that the treatment effect on conversion differs across device types. Relative to desktop, the treatment effect was significantly lower for both mobile and tablet users.

The observed conversion rates were:

Device	Control	Treatment	Difference
Desktop	7.53%	10.11%	+2.58 pp
Mobile	7.07%	5.42%	-1.66 pp
Tablet	6.61%	5.10%	-1.51 pp

The device interaction is therefore an important consideration when evaluating the redesigned checkout. The experiment suggests that the treatment response is heterogeneous by device rather than uniform across the user population.

These findings should be interpreted within the scope of the experiment period and population. Further investigation of the mobile and tablet checkout experience would be appropriate before treating the observed device differences as a general product effect.

Treatment × Device Interaction

The descriptive device analysis showed substantially different treatment effects across devices:

- **Desktop:** +2.58 percentage points
- **Mobile:** –1.66 percentage points
- **Tablet:** –1.51 percentage points

Because separate segment tests do not directly test whether the treatment effect itself differs between segments, a logistic regression with a **Variant × Device interaction** was used.

The model specification was:

```
Converted ~ Variant × Device
```

Desktop was used as the reference device.

Interaction Results

Interaction	Coefficient	p-value	Interpretation
Treatment × Mobile	-0.6072	<0.001	Treatment effect significantly lower on mobile than desktop
Treatment × Tablet	-0.5985	0.001	Treatment effect significantly lower on tablet than desktop

Both interaction terms were statistically significant.

Interpretation

The interaction model provides evidence that the effect of the redesigned checkout differs by device.

The treatment effect is substantially more positive on desktop, while the treatment effect is significantly lower on both mobile and tablet relative to desktop.

This is important because the overall experiment result alone can hide meaningful differences across user experiences.

However, these results should be interpreted within the scope of this experiment population and four-week test period. The interaction does not establish the exact cause of the device-level difference.

Business Implication

The redesigned checkout should not be treated as a uniform experience across all devices.

The strong positive desktop result combined with the weaker or negative mobile/tablet results suggests that the mobile and tablet checkout experience requires further investigation before a broad rollout.

Potential areas for investigation include:

- Mobile/tablet layout and usability
- Checkout form completion
- Page load or interaction friction
- Payment flow behavior
- CTA visibility and placement
- Device-specific technical issues
- Drop-off points within the checkout funnel

Conclusion

The interaction analysis strengthens the evidence that device type is an important factor in the experiment outcome.

Therefore, the next step should be to investigate and iterate on the mobile/tablet experience before conducting another broader experiment.

Final Launch Recommendation

Recommendation: Change & Retest

The experiment does not provide sufficiently consistent evidence to recommend a broad rollout of the redesigned checkout in its current form.

What the Experiment Showed

The redesigned checkout increased the primary conversion rate from **7.26% to 7.71%**, an absolute improvement of **+0.45 percentage points** and a relative lift of approximately **6.21%**.

However, the difference was **not statistically significant**:

- Z-statistic: **1.4998**
- p-value: **0.1337**
- 95% CI: **-0.14 pp to +1.04 pp**

The revenue-related metrics also showed positive point estimates:

- AOV increased from 63.50 to **64.57** (+1.68%)
- RPU increased from 5.13 to **5.54** (+7.98%)
- Estimated incremental monthly revenue: **+\$12,578**
- 95% RPU-based monthly impact range: approximately **-1,913 to +27,069**

The RPU difference was also not statistically significant at the 5% level ($p = 0.0889$).

Key Finding: Device-Level Heterogeneity

The most important finding was the significant interaction between treatment and device.

Device	Control Conversion	Treatment Conversion	Difference
Desktop	7.53%	10.11%	+2.58 pp
Mobile	7.07%	5.42%	−1.66 pp
Tablet	6.61%	5.10%	−1.51 pp

The treatment × mobile and treatment × tablet interaction terms were statistically significant.

This indicates that the redesigned checkout did not affect users consistently across devices.

Recommended Action

Rather than immediately launching the redesigned checkout to all users, the next experiment should focus on addressing the mobile and tablet experience.

The follow-up test should:

1. Investigate the mobile and tablet checkout funnel for points of increased abandonment.
2. Review device-specific UI, form, payment, and performance issues.
3. Preserve the successful elements of the desktop redesign.
4. Create an improved mobile/tablet version.
5. Run a new randomized A/B test with the same primary conversion metric.
6. Predefine the primary hypothesis and success criteria before the experiment begins.
7. Continue monitoring AOV, RPU, and revenue per user as secondary business metrics.
8. Pre-specify device-level analysis and the interaction test to evaluate whether the redesign performs consistently across devices.

Final Decision

Change & Retest

The experiment produced encouraging overall point estimates, but the primary conversion improvement was not statistically significant and the revenue impact has substantial uncertainty.

More importantly, the treatment effect varied significantly by device, with a strong positive desktop effect but weaker performance on mobile and tablet.

A revised checkout focused on resolving the device-specific differences should therefore be tested before making a full rollout decision.

